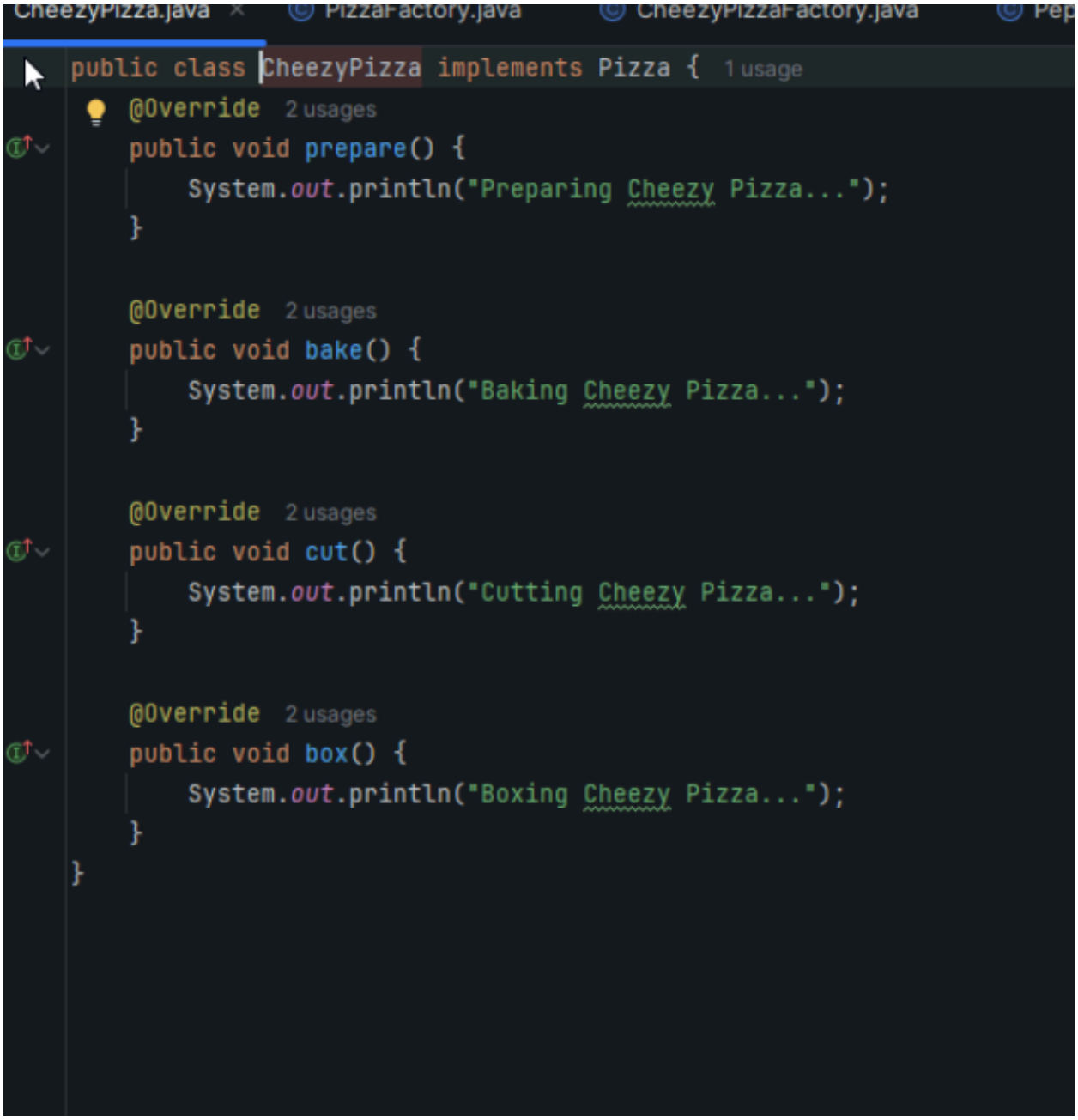
Task 01:

On the basis of above SS try to create a factory Method Design Pattern program

```java
public class FactoryMethodDesignPattern {
    public static void main(String[] args) {
        // Create Cheezy Pizza
        PizzaFactory cheezyFactory = new CheezyPizzaFactory();
        Pizza cheezyPizza = cheezyFactory.createPizza();
        cheezyPizza.prepare();
        cheezyPizza.bake();
        cheezyPizza.cut();
        cheezyPizza.box();

        System.out.println();

        // Create Pepperoni Pizza
        PizzaFactory pepperoniFactory = new PepperoniPizzaFactory();
        Pizza pepperoniPizza = pepperoniFactory.createPizza();
        pepperoniPizza.prepare();
        pepperoniPizza.bake();
        pepperoniPizza.cut();
        pepperoniPizza.box();
    }
}
```


CheezyPizza.java
PizzaFactory.java
CheezyPizzaFactory.java
public class CheezyPizza implements Pizza { 1 usage
@Override 2 usages
public void prepare() {
System.out.println("Preparing Cheezy Pizza...");
}
@Override 2 usages
public void bake() {
System.out.println("Baking Cheezy Pizza...");
}
@Override 2 usages
public void cut() {
System.out.println("Cutting Cheezy Pizza...");
}
@Override 2 usages
public void box() {
System.out.println("Boxing Cheezy Pizza...");
}
}

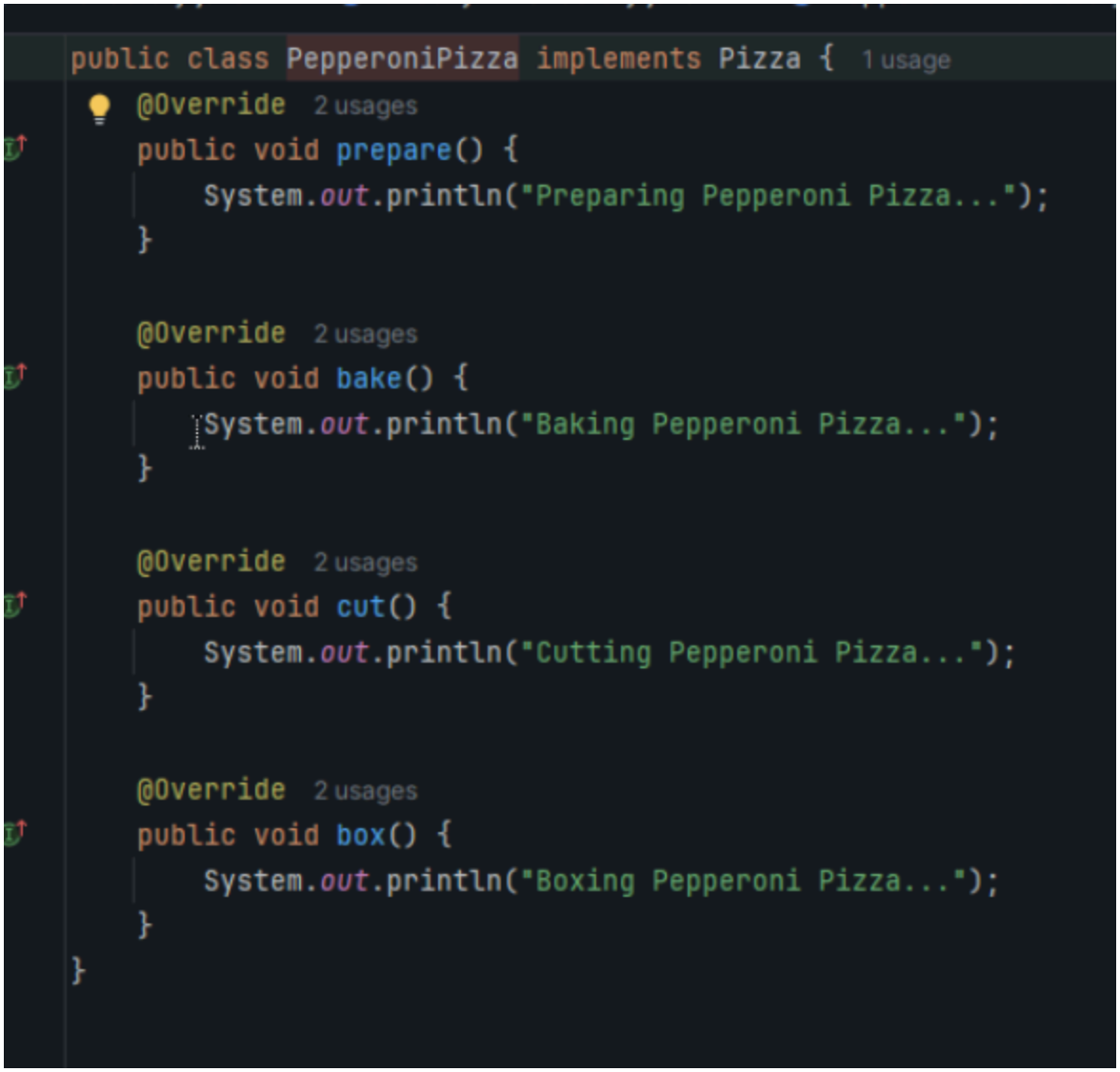
CheezyPizza.java | PizzaFactory.java | Chee

```java
public abstract class PizzaFactory {  no usag
    public abstract Pizza createPizza();  no
}
```

```java
public class CheezyPizzaFactory extends PizzaFactory {  1 usage

    @Override  2 usages
    public Pizza createPizza() {
        return new CheezyPizza();
    }
}
```

CheezyPizza.java | PizzaFactory.java | CheezyPizzaFactory.java | PepperoniPizzaFactory.java

```java
public class PepperoniPizzaFactory extends PizzaFactory {  1 usage

    @Override  2 usages
    public Pizza createPizza() {
        return new PepperoniPizza();
    }
}
```


public class PepperoniPizza implements Pizza { 1 usage
@Override 2 usages
public void prepare() {
System.out.println("Preparing Pepperoni Pizza...");
}
@Override 2 usages
public void bake() {
System.out.println("Baking Pepperoni Pizza...");
}
@Override 2 usages
public void cut() {
System.out.println("Cutting Pepperoni Pizza...");
}
@Override 2 usages
public void box() {
System.out.println("Boxing Pepperoni Pizza...");
}
}

PizzaFactory.java CheezyPizzaFactory.java PepperoniPizzaFac

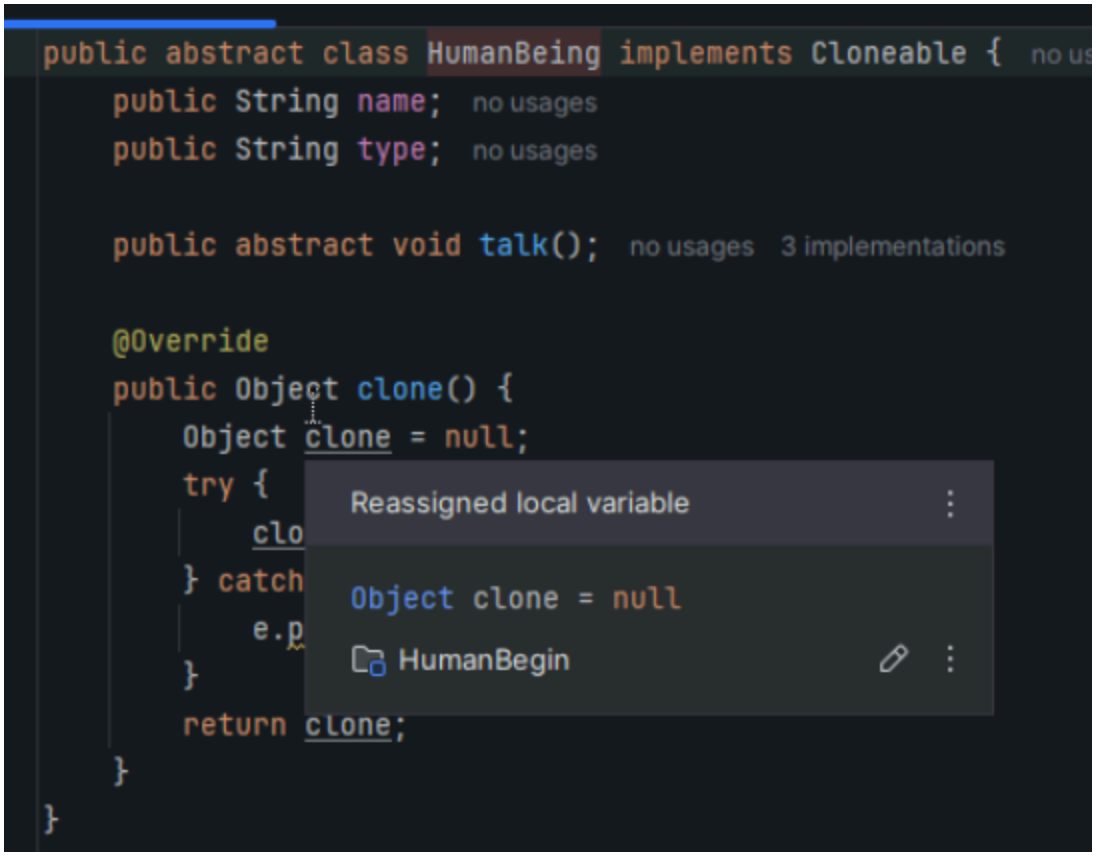
```java
public class PepperoniPizzaFactory extends PizzaFactory {
    @Override
    public Pizza createPizza() {
        return new PepperoniPizza();
    }
}
```

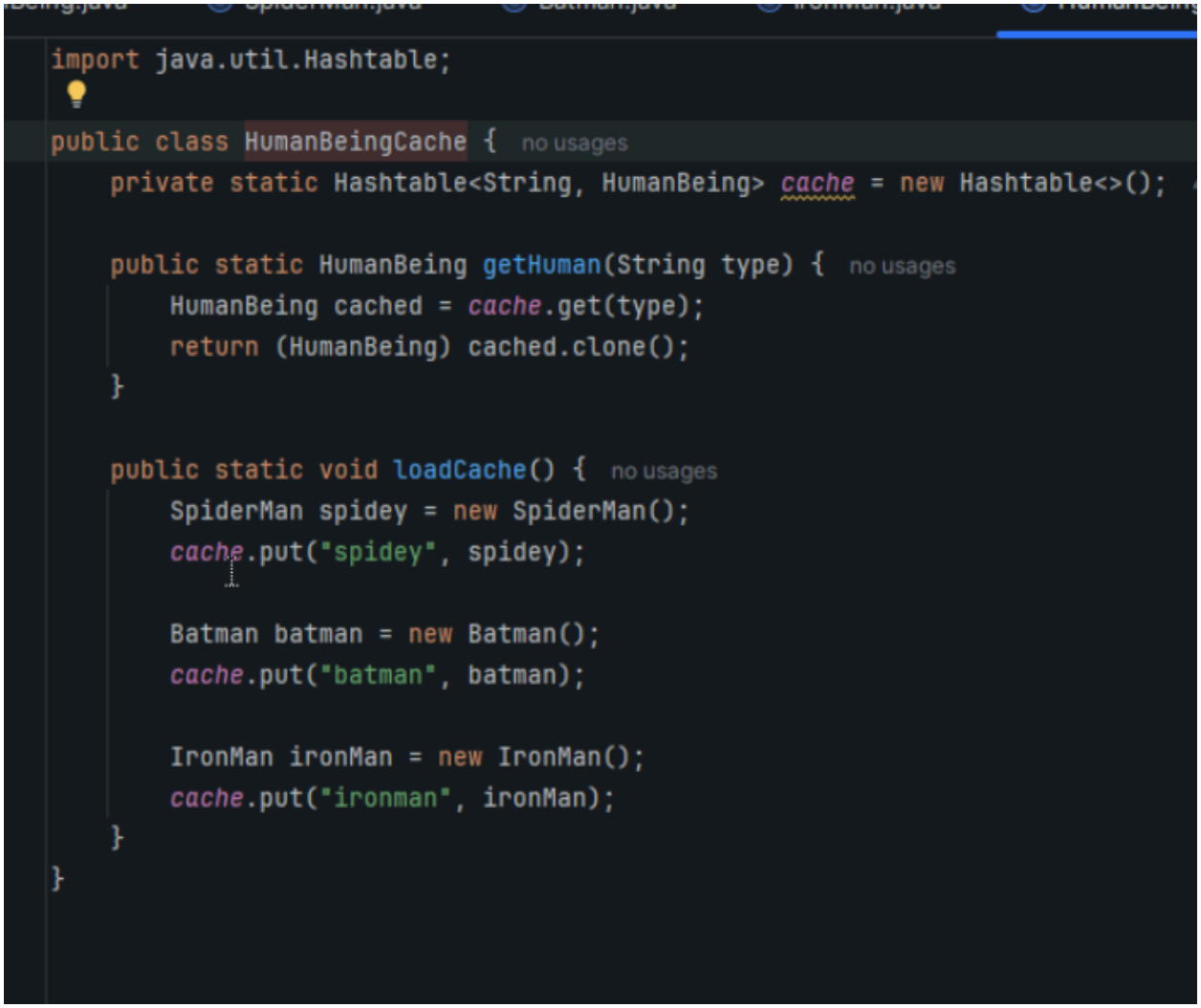
```java
public interface Pizza {
    void prepare();
    void bake();
    void cut();
    void box();
}
```

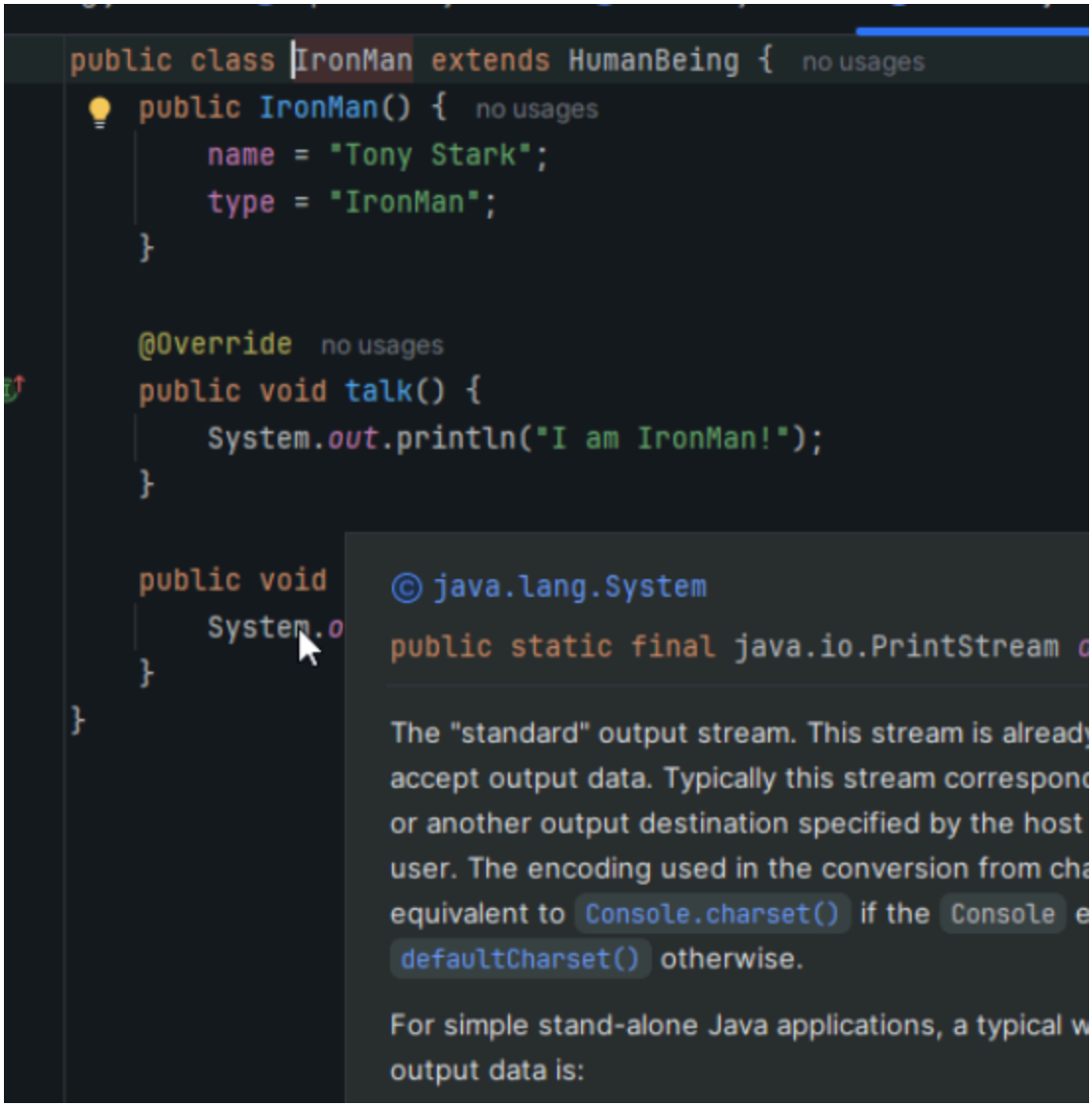
```java
public abstract class PizzaFactory {
    public abstract Pizza createPizza();
}
```

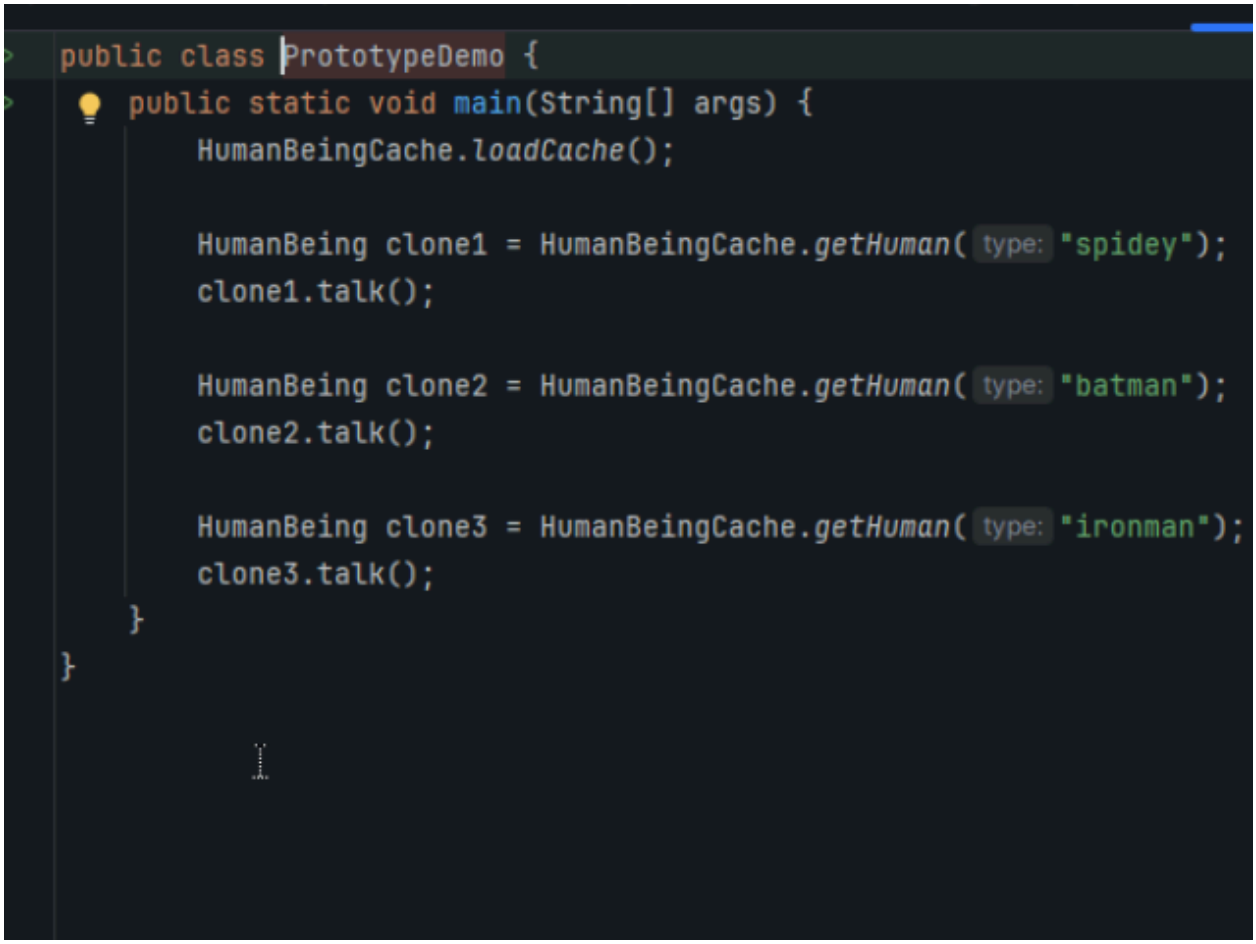
```
"C:\Program Files\Java\jdk-17\bin\j
Preparing Cheezy Pizza...
Baking Cheezy Pizza...
Cutting Cheezy Pizza...
Boxing Cheezy Pizza...

Preparing Pepperoni Pizza...
Baking Pepperoni Pizza...
Cutting Pepperoni Pizza...
Boxing Pepperoni Pizza...

Process finished with exit code 0
```

Task4


public abstract class HumanBeing implements Cloneable {
public String name; no usages
public String type; no usages
public abstract void talk(); no usages 3 implementations
@Override
public Object clone() {
Object clone = null;
try {
} catch
e.p
}
return clone;
}
}
Reassigned local variable
Object clone = null
HumanBegin


import java.util.Hashtable;
public class HumanBeingCache { no usages
private static Hashtable<String, HumanBeing> cache = new Hashtable<>();
public static HumanBeing getHuman(String type) { no usages
HumanBeing cached = cache.get(type);
return (HumanBeing) cached.clone();
}
public static void loadCache() { no usages
SpiderMan spidey = new SpiderMan();
cache.put("spidey", spidey);
Batman batman = new Batman();
cache.put("batman", batman);
IronMan ironMan = new IronMan();
cache.put("ironman", ironMan);
}
}


public class IronMan extends HumanBeing { no usages
public IronMan() { no usages
name = "Tony Stark";
type = "IronMan";
}
@Override no usages
public void talk() {
System.out.println("I am IronMan!");
}
public void
System.o
}
}
java.lang.System
public static final java.io.PrintStream
The "standard" output stream. This stream is alread
accept output data. Typically this stream correspon
or another output destination specified by the host
user. The encoding used in the conversion from ch
equivalent to Console.charset() if the Console
defaultCharset() otherwise.
For simple stand-alone Java applications, a typical w
output data is:


public class PrototypeDemo {
public static void main(String[] args) {
HumanBeingCache.loadCache();
HumanBeing clone1 = HumanBeingCache.getHuman(type: "spidey");
clone1.talk();
HumanBeing clone2 = HumanBeingCache.getHuman(type: "batman");
clone2.talk();
HumanBeing clone3 = HumanBeingCache.getHuman(type: "ironman");
clone3.talk();
}
}

```java
public class SpiderMan extends HumanBeing {  no usages
    public SpiderMan() {  no usages
        name = "Peter Parker";
        type = "SpiderMan";
    }

    @Override  no usages
    public void talk() {
        System.out.println("I am SpiderMan!");
    }

    public void webShoot() {  no usages
        System.out.println("Shooting webs...");
    }
}
```

```
"C:\Program Files\Java\jdk-
I am SpiderMan!
I am Batman!
I am IronMan!

Process finished with exit
```

egin > src > PrototypeDemo